

Contents

1 Basic	1
1.1 標準輸入	1
1.2 標準輸出	1
1.3 運算子	1
1.4 型別檢查	2
1.5 其他	2
2 Data Structure	2
2.1 列表 List	2
2.2 元組 Tuple	3
2.3 集合 Set	4
2.4 字典 Dict	5
2.5 字串 String	5
2.6 堆疊 Stack	7
2.7 佇列 Queue	7
2.8 鏈結串列 Link List	8
2.9 二元樹 Binary Tree	8
3 函式 Function	8
3.1 你該知道的事 Things You Should Know	8
3.2 位置參數與關鍵字參數 Positional and Keyword Arguments	8
3.3 可變長度參數 Variable-length Arguments	8
3.4 高階函式 Higher-Order Functions	9
3.5 閉包 Closure	9
3.6 Lambda 函式 Lambda Functions	9
3.7 內建高階函式 Built-in Higher-Order Functions	9
3.8 裝飾器 Decorators	9
3.9 迭代器 Iterators	10
3.10 產生器 Generators	10
4 File Handling	10
4.1 基本檔案操作 Basic File Operations	10
5 Exception Handling	11
5.1 基本例外處理 Basic Exception Handling	11
5.2 else 和 finally 子句 else and finally Clauses	11
5.3 自訂例外 Custom Exceptions	11
5.4 斷言 Assertions	11
6 Object-Oriented	11
6.1 建立類別 Class Declaration	11
6.2 類別的方法 Class Methods	11
6.3 建構子跟解構子 Constructor and Destructor	12
6.4 封裝 Encapsulation	12
6.5 封裝屬性 Encapsulation with Property	12
6.6 繼承 Inheritance	12
6.7 多型 Polymorphism	12
7 Algorithm	12
7.1 矩陣操作 Matrix Operations	12
7.2 Sort: Bubble Sort	13
7.3 Sort: Selection Sort	13
7.4 Sort: Insertion Sort	13
7.5 Sort: Quick Sort	13
7.6 Sort: Quick Sort(In Place)	13
7.7 Sort: Merge Sort	13
7.8 Search: Binary Search	13
7.9 String: KMP	13

1 Basic

1.1 標準輸入

```
# 輸入多個變數 (字串)
x, y, z = input().split()
# 輸入多個變數 (自動轉型整數)
x, y, z = map(int, input().split())
# 輸入一個 list (array) (字串)
arr = input().split()
# 輸入一個 list (array) (自動轉型整數)
arr = list(map(int, input().split()))
# 輸入一個多維 list (array) (字串)
matrix = []
n = 3 # 假設有 n 行
for i in range(n):
    row = input().split()
    matrix.append(row)
# 輸入一個多維 list (array) (自動轉型整數)
matrix = []
n = 3 # 假設有 n 行
for i in range(n):
    row = list(map(int, input().split()))
    matrix.append(row)
# 輸入一串指令 (有整數有字串) (以 HW 6-1 為例)
miku = list()
getline = str(input())
cmd = getline.split()

if not cmd:
    continue # optional: skip this time
```

```
oper = cmd[0]
```

```
if oper == "stop":
    break
```

```
elif oper == "print":
    for i in miku:
        i.output()
```

```
else:
    in_band = cmd[1]
    in_year = cmd[2]
    #... 自行處理
```

```
# 輸入到 EOF
```

```
try:
    while True:
        n = int(input())
        # process n
except EOFError:
    pass
```

1.2 標準輸出

```
# 輸出多個變數
```

```
a, b, c = 1, 2, 3
```

```
print(a, b, c) # 預設以空白分隔
```

```
print(a, b, c, sep='#') # 改以 # 分隔
```

```
# 格式化輸出
```

```
name = "Alice"
```

```
age = 30
```

```
print(f"Name:{name}, Age:{age}")
```

```
# 控制浮點數輸出
```

```
pi = 3.141592653589793
```

```
print(f"Pi: {pi:.2f}") # 保留小數點後兩位
```

```
# 控制輸出不換行
```

```
print("Hello Kitty", end=' ') # 以空格結束，不換行
```

```
print("Hello World", end="") # 以空字串結束，不換行
```

```
# 字串陣列輸出
```

```
uwu = ['a', 'b', 'c', 'd', 'e']
```

```
print(f"uwu: {' '.join(uwu)}")
```

```
print(*uwu) # 解包輸出陣列元素，預設以空白分隔
```

```
print(*uwu, sep="##") # 解包輸出陣列元素，以 ## 分隔
```

```
# 整數陣列格式化輸出
```

```
num = [1, 2, 3, 4, 5]
```

```
print(*num)
```

```
# 多維字串陣列輸出
```

```
matrix_str = [ ['a', 'b', 'c'], ['d', 'e', 'f'], ['g', 'h', 'i'] ]
```

```
for row in matrix_str:
```

```
    print(*row)
```

```
# 多維整數陣列輸出
```

```
matrix_int = [ [1,2,3], [4,5,6], [7,8,9] ]
```

```
for row in matrix_int:
```

```
    print(*row)
```

1.3 運算子

```
# 算數運算
```

```
a = 10
```

```
b = 3
```

```
print(a + b) # Addition: 13
```

```
print(a - b) # Subtraction: 7
```

```
print(a * b) # Multiplication: 30
```

```
print(a / b) # Division: 3.333...
```

```
print(a // b) # Floor Division: 3
```

```
print(a % b) # Modulus: 1
```

```
print(a ** b) # Exponentiation: 1000
```

```
# 比較運算
```

```
a = 10
```

```
b = 3
```

```
print(a == b) # Equal: False
```

```
print(a != b) # Not equal: True
```

```
print(a < b) # Less than: True
```

```
print(a > b) # Greater than: False
```

```
print(a <= b) # Less than or equal: True
```

```
print(a >= b) # Greater than or equal: False
```

```
print("apple" < "banana") # 字串比較(比較字典序), True
```

```
# 邏輯運算
```

```
x = True
```

```
y = False
```

```
print(x and y) # AND: False
print(x or y) # OR: True
print(not x) # NOT: False

# 位元運算
i = 10 # Binary: 1010
j = 4 # Binary: 0100
print(i & j) # AND: 0 (0000)
print(i | j) # OR: 14 (1110)
print(i ^ j) # XOR: 14 (1110)
print(~i) # NOT: -11
print(i << 1) # Left shift: 20
print(i >> 1) # Right shift: 5
```

1.4 型別檢查

```
# type() 型別檢查
uwu = 69
print(type(uwu)) # <class 'int'>
print(type(uwu) == int) # True

# isinstance() 型別檢查
owo = 87
print(isinstance(owo, int)) # True
print(isinstance(owo, str)) # False
a = list()
print(isinstance(a, tuple)) # False
print(isinstance(a, list)) # True

# 檢查 tuple (或一組資料) 內的所有值
## isinstance 可以檢查多重型別
b = (777, "miku", 3.14)
print(all(isinstance(x, (int, str, float)) for x in b))
# True

# 檢查物件繼承關係
## isinstance 可以檢查繼承關係或是物件
class Animal:
    pass

class Dog(Animal): # Dog 繼承自 Animal
    pass
print(type(Dog()) == Dog) # True
print(type(Dog()) == Animal) # False
print(isinstance(Dog(), Dog)) # True
print(isinstance(Dog(), Animal)) # True
```

1.5 其他

```
# 三元運算
## 也可以不只三元，你可以自己往下加，如果你想讓你的
Code 看起來像義大利麵的話不是不行
## result = TRUE_COND if condition else FALSE_COND

age = 20
isAdult = True if age >= 18 else False
## 上方寫法等價
if age >= 18:
    isAdult = True
else:
    isAdult = False

# range() 函數
range(STOP) # 從 0 到 STOP-1
range(START, STOP)
# 從 START 到 STOP-1 (預設間隔 1)
range(START, STOP, STEP)
# 從 START 到 STOP-1，間隔 STEP
```

2 Data Structure

2.1 列表 List

請注意 Python List 的 Index: alphabet = ['a', 'b', 'c'] - POS_Index: 0, 1, 2 - NEG_Index: -3, -2, -1

```
# Python 的 List 允許將不同型別的資料存於同列表中
grades = [85, 92, 78, 96, 89]
students = ["Alice", "Bob", "Charlie"]
mixed_data = [1, "hello", 3.14, True]
```

```
# 初始化
uwu = list()
```

```
uwu = []
uwu = [0] * 5 # [0, 0, 0, 0, 0]
uwu = [1, 2] * 3 # [1, 2, 1, 2, 1, 2]
uwu = [x**2 for x in range(5)] # [0, 1, 4, 9, 16]
```

```
# List 可用函數
len(uwu) # 取得長度
sum(uwu) # 計算總和
max(uwu) # 取得最大值
min(uwu) # 取得最小值
del uwu[0] # 刪除指定位置的元素
del uwu[1:3] # 刪除指定index的元素
```

```
# List 可用方法
uwu.append(100) # 新增元素
uwu.count(100) # 計算指定元素出現次數
uwu.extend([200, 300]) # 在list的尾部添加多個元素
uwu.index(100) # 回傳指定元素的索引位置
uwu.insert(0, 50) # 在指定位置插入元素 (index, element)
uwu.pop() # 移除並回傳指定位置的元素 (預設為最後一個，
也可以指定 index)
uwu.remove(100) # 移除指定元素 (第一個出現的)
uwu.reverse() # 將列表反轉
uwu.sort() # 將列表排序 (預設由小到大，加入參數 reverse
=True 可由大到小)
```

```
# 自定義排序方法
def get_grade(student_tuple):
    return student_tuple[1]
```

```
students = [("Alice", 85), ("Bob", 92), ("Charlie", 78)]
students.sort(key=get_grade) # 依照成績排序
students.sort(key=lambda x: x[1]) # 使用 lambda (效果
同上)
```

```
# 複製 List
## 一維 List
original_list = [1, 2, 3, 4, 5]
copy_list = original_list.copy() # 使用 copy 方法複製
copy_list2 = original_list[:] # 使用切片複製
wrong_copy_list3 = original_list # 複製參考而已，改了其
中一方會影響另一方
## 多維 List
import copy
original_2d_list = [[1, 2, 3], [4, 5, 6]]
deep_copy_2d = copy.deepcopy(original_2d_list) # 請使用
deepcopy
wrong_copy_2d = original_2d_list.copy() # 只會複製最外
層，內層仍是參考
```

```
# String to List
sentence = "Hello, world!"
char_list = list(sentence) # 將字串轉換為字元列表
```

```
# List Slicing
## [start:stop:step] , start 包含，stop 不包含，step 為
步進，預設為 1
numbers = [10, 20, 30, 40, 50, 60]
numbers[0:3] # 取得索引 0 到 2 的元素
numbers[3:] # 取得索引 3 到結尾的元素
numbers[:4] # 取得從開始到索引 3 的元素
numbers[-1] # 取得最後一個元素
numbers[-3:] # 取得最後三個元素
numbers[:2] # 取得所有偶數索引的元素
numbers[::-1] # 反轉列表
```

```
# List Comprehension
## 初始化，或是存值
numbers = [i for i in range(1, 6)]
# [1, 2, 3, 4, 5]
pairs = [(x, y) for x in range(3) for y in range(2)]
# [(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1)]
```

```
## 對元素操作
names = ["alice", "bob", "charlie"]
upper_names = [name.upper() for name in names] # ['
ALICE', 'BOB', 'CHARLIE']
```

```
## 對元素計算
numbers = [1, 2, 3, 4, 5]
```

```
squared = [x**2 for x in numbers]
# [1, 4, 9, 16, 25]

## 解包
students = [("Alice", 85), ("Bob", 92)]
names = [name for name, grade in students] # ['Alice', 'Bob']
grades = [grade for name, grade in students] # [85, 92]

## 帶條件篩選 (配合三元運算子)
evens = [x for x in range(20) if x % 2 == 0] # [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
special = [x for x in numbers if x > 3 and x < 8] # [4, 5, 6, 7]

## 範例：算平均 / 找最高值
stu = [{"name": "Alice", "scores": [85, 92, 78]}, {"name": "Bob", "scores": [92, 88, 95]}]
avg = [sum(s["scores"])/len(s["scores"]) for s in stu]
# [85.0, 91.66666666666667]
high_performers = [s["name"] for s in stu if sum(s["scores"])/len(s["scores"]) >= 90] # ['Bob']
```

2.2 元組 Tuple

請注意 Python 的 tuple 是不可變的 (immutable)。但 tuple 的內容可以包含可變更的物件 (例如 list)。請注意 Python Tuple 的 Index: alphabet = ('a', 'b', 'c') - POS_Index: 0, 1, 2 - NEG_Index: -3, -2, -1

```
# 初始化
uwu = tuple()
uwu = ()
uwu = (1,) # 單元素 tuple 需要加逗號
uwu = tuple([1, 2, 3]) # 從 list 建立
uwu = tuple("hello") # 從字串建立: ('h', 'e', 'l', 'l', 'o')
uwu = tuple(range(5)) # (0, 1, 2, 3, 4)
uwu = tuple(x**2 for x in range(5)) # tuple 推導式 (0, 1, 4, 9, 16)
person = ("Alice", 20, "Student") # 可包含不同型態的變數
mixed = (1, "hello", 3.14, [1, 2, 3]) # 可包含 list 等可變物件

# Tuple 可用函數
len(uwu) # 取得長度
max(uwu) # 取得最大值 (元素必須可比較)
min(uwu) # 取得最小值
sum(uwu) # 計算總和 (元素必須是數字)

# Tuple 可用方法
uwu.count(1) # 計算指定元素出現次數
uwu.index(1) # 回傳指定元素的第一個索引位置 (找不到會 ValueError)

# 存取元素
numbers = (10, 20, 30, 40, 50)
numbers[0] # 第一個元素: 10
numbers[-1] # 最後一個元素: 50
numbers[1:3] # 切片: (20, 30)

# Tuple Unpacking
point = (3, 4)
x, y = point # x=3, y=4

person = ("Alice", 20, "Student")
name, age, role = person # name="Alice", age=20, role="Student"

## 使用 * 收集剩餘元素
numbers = (1, 2, 3, 4, 5)
first, *rest = numbers # first=1, rest=[2, 3, 4, 5]
first, *middle, last = numbers # first=1, middle=[2, 3, 4], last=5
*head, last = numbers # head=[1, 2, 3, 4], last=5

## 交換變數
a, b = 1, 2
a, b = b, a # a=2, b=1

## 忽略某些值
person = ("Alice", 20, "Student", "CS")
```

```
name, _, role, _ = person # 使用 _ 表示忽略的值
```

```
# Tuple Slicing
## 與 list 相同的切片語法
numbers = (10, 20, 30, 40, 50, 60)
numbers[0:3] # (10, 20, 30)
numbers[3:] # (40, 50, 60)
numbers[:4] # (10, 20, 30, 40)
numbers[-3:] # (40, 50, 60)
numbers[::2] # (10, 30, 50)
numbers[::-1] # (60, 50, 40, 30, 20, 10)
```

```
# 迭代 Tuple
fruits = ("apple", "banana", "orange")
for fruit in fruits:
    print(fruit)
```

```
## 使用 enumerate 取得索引和值
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")
```

```
# 檢查元素是否存在
numbers = (1, 2, 3, 4, 5)
if 3 in numbers:
    print("3 is in the tuple")
if 10 not in numbers:
    print("10 is not in the tuple")
```

```
# Tuple 串接和重複
tuple1 = (1, 2, 3)
tuple2 = (4, 5, 6)
combined = tuple1 + tuple2 # (1, 2, 3, 4, 5, 6)
repeated = tuple1 * 3 # (1, 2, 3, 1, 2, 3, 1, 2, 3)
```

```
# Named Tuple
from collections import namedtuple
```

```
Point = namedtuple('Point', ['x', 'y'])
p = Point(3, 4)
print(p.x, p.y) # 3 4
print(p[0], p[1]) # 也可以用索引: 3 4
```

```
Student = namedtuple('Student', ['name', 'age', 'grade'])
student = Student('Alice', 20, 'A')
print(student.name) # Alice
print(student.age) # 20
```

```
## namedtuple 的方法
student._asdict() # 轉換為 OrderedDict
student._replace(age=21) # 建立新的 namedtuple, 修改指定欄位
Student._fields # 取得所有欄位名稱: ('name', 'age', 'grade')
```

```
## 從 dict 建立 namedtuple
data = {'name': 'Bob', 'age': 22, 'grade': 'B'}
student2 = Student(**data)
```

```
# Tuple 作為 Dictionary 的 Key
## 因為 tuple 不可變, 可以作為 dict 的 key (list 不行)
locations = {
    (0, 0): "Origin",
    (1, 0): "East",
    (0, 1): "North"
}
print(locations[(0, 0)]) # Origin
```

```
## 多維座標
grid = {
    (0, 0, 0): "Center",
    (1, 0, 0): "Right",
    (0, 1, 0): "Up"
}
```

```
# Tuple 可包含可變物件
## Tuple 本身不可變, 但可以包含可變物件
t = (1, 2, [3, 4])
t[0] = 100 # 不能改 tuple 的元素
t[2].append(5) # 可以修改 list 的內容: (1, 2, [3, 4, 5])
t[2][0] = 30 # 可以修改 list 的元素: (1, 2, [30, 4, 5])
```

```
## Tuple 不能作為 dict 的 key
d = {(1, 2, [3, 4]): "value"} # 錯誤! TypeError:
    unhashable type: 'list'

# Sample Applications
## 函數回傳多個值
def get_stats(numbers):
    return min(numbers), max(numbers), sum(numbers)

numbers = [1, 2, 3, 4, 5]
min_val, max_val, total = get_stats(numbers)

## 函數參數解包
def add(a, b, c):
    return a + b + c

numbers = (1, 2, 3)
result = add(*numbers) # 6

## 同時迭代多個序列
names = ("Alice", "Bob", "Charlie")
scores = (85, 92, 78)
for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

2.3 集合 Set

請注意 Python 的 Set 是無序的 (unordered)。set 的內容也不可以包含可變的物件 (例如 list)。

```
# 初始化
uwu = set()
uwu = {1, 2, 3}
fruits = {"apple", "banana", "orange"}
uwu = set([1, 2, 3, 2, 1]) # 從 list 建立，自動去重:
    {1, 2, 3}
uwu = set("hello") # 從字串建立: {'h', 'e', 'l', 'o'}
uwu = {x for x in range(5)} # Set 推導式: {0, 1, 2, 3, 4}
mixed = {1, "hello", 3.14, (1, 2)} # 可以混合類型，但不能包含 list 或 dict

# Set 可用函數
len(uwu) # 取得元素數量
max(uwu) # 取得最大值 (元素必須可比較)
min(uwu) # 取得最小值
sum(uwu) # 計算總和 (元素必須是數字)

# Set 可用方法
uwu.add(10) # 新增單個元素
uwu.clear() # 清空集合
uwu.copy() # 淺複製
uwu.discard(10) # 移除元素 (元素不存在也不會報錯)
uwu.remove(10) # 移除元素 (元素不存在會引發 KeyError)
uwu.pop() # 隨機移除並回傳一個元素 (set 無序，所以是"隨機")
uwu.update({6, 7, 8}) # 新增多個元素 (可接受 set, list, tuple 等可迭代物件)
uwu.update([9, 10], {11, 12}) # 可同時更新多個可迭代物件

# 集合運算
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

## 聯集 (Union) - 所有元素
union1 = set1 | set2 # {1, 2, 3, 4, 5, 6}
union2 = set1.union(set2) # 同上
set1 |= set2 # 原地更新為聯集

## 交集 (Intersection) - 共同元素
inter1 = set1 & set2 # {3, 4}
inter2 = set1.intersection(set2) # 同上
set1 &= set2 # 原地更新為交集

## 差集 (Difference) - 在 set1 但不在 set2
diff1 = set1 - set2 # {1, 2}
diff2 = set1.difference(set2) # 同上
set1 -= set2 # 原地更新為差集

## 對稱差集 (Symmetric Difference) - 在其中一個但不在兩者
```

```
sym_diff1 = set1 ^ set2 # {1, 2, 5, 6}
sym_diff2 = set1.symmetric_difference(set2) # 同上
set1 ^= set2 # 原地更新為對稱差集

# 集合關係判斷
set1 = {1, 2, 3}
set2 = {1, 2, 3, 4, 5}
set3 = {1, 2, 3}

set1.issubset(set2) # set1 是否為 set2 的子集: True
set1 <= set2 # 同上: True
set1 < set2 # 是否為真子集 (不相等的子集): True

set2.issuperset(set1) # set2 是否為 set1 的父集: True
set2 >= set1 # 同上: True
set2 > set1 # 是否為真父集: True

set1.isdisjoint(set2) # 是否沒有交集: False
set1 == set3 # 是否相等: True

# 檢查元素是否存在
numbers = {1, 2, 3, 4, 5}
if 3 in numbers:
    print("3 is in the set")
if 10 not in numbers:
    print("10 is not in the set")

# 迭代 Set (順序不固定)
fruits = {"apple", "banana", "orange"}
for fruit in fruits:
    print(fruit)

# 複製 Set
original = {1, 2, 3}
copy_set = original.copy()
copy_set2 = set(original)
wrong_copy = original # 只是複製參考

# Set Comprehension
## 建立集合
squares = {x**2 for x in range(6)} # {0, 1, 4, 9, 16, 25}
evens = {x for x in range(10) if x % 2 == 0} # {0, 2, 4, 6, 8}

## 從字串提取唯一字符
text = "hello world"
unique_chars = {char for char in text if char != ' '} # {'h', 'e', 'l', 'o', 'w', 'r', 'd'}

## 處理多個集合
lists = [[1, 2], [2, 3], [3, 4]]
all_numbers = {num for lst in lists for num in lst} # {1, 2, 3, 4}

# Frozenset 不可變的 Set (immutable)
## frozenset 可以作為 dict 的 key 或 set 的元素
fs = frozenset([1, 2, 3])
regular_set = {5, 6, 7, 8}
fs2 = frozenset(regular_set)

## frozenset 支援所有不會修改集合的操作
fs1 = frozenset([1, 2, 3])
fs2 = frozenset([3, 4, 5])
union = fs1 | fs2 # frozenset({1, 2, 3, 4, 5})
inter = fs1 & fs2 # frozenset({3})

## 用途: 作為 dict 的 key
locations = {
    frozenset(["A", "B"]): "Route 1",
    frozenset(["B", "C"]): "Route 2"
}

# Sample Applications
## 去除重複
numbers = [1, 2, 2, 3, 3, 3, 4, 5, 5]
unique = list(set(numbers)) # [1, 2, 3, 4, 5] (順序可能不同)

## 找出共同元素
list1 = [1, 2, 3, 4]
list2 = [3, 4, 5, 6]
```

```
common = list(set(list1) & set(list2)) # [3, 4]

## 找出唯一元素 (只在一個列表中)
unique_to_list1 = list(set(list1) - set(list2)) # [1, 2]
unique_elements = list(set(list1) ^ set(list2)) # [1, 2, 5, 6]

## 移除列表中的特定元素
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
to_remove = {2, 4, 6, 8}
filtered = [x for x in numbers if x not in to_remove] # [1, 3, 5, 7, 9]

## 檢查是否有重複
def has_duplicates(lst):
    return len(lst) != len(set(lst))

items = [1, 2, 3, 2]
print(has_duplicates(items)) # True
```

2.4 字典 Dict

```
# 初始化
uwu = dict()
uwu = {}
uwu = dict([("a", 1), ("b", 2)]) # 從 tuple list 建立
uwu = dict(a=1, b=2, c=3) # 使用關鍵字參數
uwu = {k: v for k, v in [("x", 1), ("y", 2)]} # 字典推導式
uwu = dict.fromkeys(["a", "b", "c"], 0) # 用相同值初始化多個 key: {'a': 0, 'b': 0, 'c': 0}
## Sample Usage
student_scores = {"Alice": 85, "Bob": 92, "Charlie": 78}
ages = {"Alice": 20, "Bob": 21}
mixed_dict = {"name": "Alice", "age": 20, "scores": [85, 92, 78], "is_active": True}

# Dictionary 可用函數
len(uwu) # 取得鍵值對數量
max(uwu) # 取得最大的 key (依字母或數字排序)
min(uwu) # 取得最小的 key
del uwu["a"] # 刪除指定 key 的鍵值對

# Dictionary 可用方法
uwu.clear() # 清空字典
uwu.copy() # 淺複製字典
uwu.get("key", "default") # 取得值, key 不存在時回傳 default (預設為 None)
uwu.items() # 回傳所有鍵值對的視圖 (可用於迭代)
uwu.keys() # 回傳所有 key 的視圖
uwu.values() # 回傳所有 value 的視圖
uwu.pop("key", "default") # 移除並回傳指定 key 的值, 不存在時回傳 default
uwu.popitem() # 移除並回傳最後一個鍵值對 (Python 3.7+ 保證順序)
uwu.setdefault("key", "default") # 如果 key 存在則回傳值, 否則設定為 default 並回傳
uwu.update({"new_key": "value"}) # 更新字典 (可接受字典、鍵值對列表或關鍵字參數)

# 存取與修改
scores = {"Alice": 85, "Bob": 92}
scores["Alice"] # 取得值 (key 不存在會引發 KeyError)
scores.get("Charlie", 0) # 安全取得值, 不存在回傳 0
scores["Charlie"] = 78 # 新增或修改
scores["Alice"] += 5 # 修改現有值

# 檢查 key 是否存在
if "Alice" in scores:
    print("Alice is in the dictionary")
if "David" not in scores:
    print("David is not in the dictionary")

# 迭代 Dictionary
student_scores = {"Alice": 85, "Bob": 92, "Charlie": 78}

## 迭代 keys
for name in student_scores:
```

```
    print(name)
for name in student_scores.keys():
    print(name)

## 迭代 values
for score in student_scores.values():
    print(score)

## 迭代 key-value pairs
for name, score in student_scores.items():
    print(f"{name}: {score}")

# 複製 Dictionary
original = {"a": 1, "b": 2}
shallow_copy = original.copy() # 淺複製
shallow_copy2 = dict(original) # 另一種淺複製方式
wrong_copy = original # 只是複製參考
## 完全複製
import copy
nested = {"a": [1, 2, 3], "b": [4, 5, 6]}
deep_copy = copy.deepcopy(nested)

# Dictionary Comprehension
## 建立字典
squares = {x: x**2 for x in range(5)} # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
names = ["Alice", "Bob", "Charlie"]
name_lengths = {name: len(name) for name in names} # {'Alice': 5, 'Bob': 3, 'Charlie': 7}

## 轉換或過濾
scores = {"Alice": 85, "Bob": 92, "Charlie": 78, "David": 95}
high_scores = {k: v for k, v in scores.items() if v >= 90} # {'Bob': 92, 'David': 95}
uppercase_keys = {k.upper(): v for k, v in scores.items()} # {'ALICE': 85, 'BOB': 92, ...}

## 反轉鍵值對
original = {"a": 1, "b": 2, "c": 3}
reversed_dict = {v: k for k, v in original.items()} # {1: 'a', 2: 'b', 3: 'c'}

## 合併字典
dict1 = {"a": 1, "b": 2}
dict2 = {"b": 3, "c": 4}
merged = dict1 | dict2 # {'a': 1, 'b': 3, 'c': 4} (後者覆蓋前者)
dict1 |= dict2 # 原地合併

# 排序 Dictionary
scores = {"Alice": 85, "Bob": 92, "Charlie": 78}

## 依 key 排序
sorted_by_key = dict(sorted(scores.items())) # {'Alice': 85, 'Bob': 92, 'Charlie': 78}

## 依 value 排序
sorted_by_value = dict(sorted(scores.items(), key=lambda x: x[1])) # {'Charlie': 78, 'Alice': 85, 'Bob': 92}
sorted_by_value_desc = dict(sorted(scores.items(), key=lambda x: x[1], reverse=True)) # 降序
```

2.5 字串 String

請注意 Python 的 String 其實是不可變的 (immutable)。任何對 string 的修改本質上其實是創建一個新的字串物件請注意 Python String 的 Index: alphabet = "a b c" (忽略空格) - POS_Index: 0 1 2 - NEG_Index: -3 -2 -1

```
# 初始化
uwu = str()
uwu = ""
uwu = "Hello"
uwu = 'Hello'
uwu = '''Triple quotes'''
uwu = """Triple quotes"""
uwu = str(123) # 將其他類型轉換為字串: "123"
uwu = "Hello " * 3 # 重複字串: "Hello Hello Hello "

# 三引號字串 (可跨多行)
text = """
This is a multi-line string.
```

```
It can span multiple lines.
Useful for long text or docstrings.
"""
```

```
# 原始字串 (Raw String) - 忽略跳脫字元
normal = "C:\\Users\\name\\file.txt" # 需要雙反斜線
raw = r"C:\Users\name\file.txt" # 使用 r 前綴, 單反斜線
# 即可
regex = r"\d+\.\d+" # 正規表達式常用
```

```
# 字串格式化
name = "Alice"
age = 20
score = 95.5
```

```
## 1. f-string (Python 3.6+) - 推薦使用
text = f"My name is {name} and I am {age} years old."
text = f"Score: {score:.2f}" # 格式化浮點數: "Score:
95.50"
text = f"{name.upper()}" # 可以包含運算式: "ALICE"
text = f"{2 + 3}" # 運算: "5"
```

```
## 2. format() 方法
text = "My name is {} and I am {} years old.".format(
    name, age)
text = "My name is {0} and I am {1} years old.".format(
    name, age) # 使用索引
text = "My name is {n} and I am {a} years old.".format(
    n=name, a=age) # 使用關鍵字
text = "Score: {:.2f}".format(score) # 格式化
```

```
## 3. % 格式化 (舊式, 不推薦)
text = "My name is %s and I am %d years old." % (name,
    age)
text = "Score: %.2f" % score
```

```
## f-string 進階用法
x = 10
text = f"{x:05d}" # 補零到5位數: "00010"
text = f"{x:>10}" # 右對齊, 寬度10: "          10"
text = f"{x:<10}" # 左對齊: "10          "
text = f"{x:^10}" # 置中: "      10      "
```

```
# String 可用函數
len("Hello") # 取得長度: 5
max("Hello") # 取得最大字元 (依 ASCII): 'o'
min("Hello") # 取得最小字元: 'H'
```

```
# String 可用方法 - 大小寫轉換
text = "Hello World"
text.upper() # 全部大寫: "HELLO WORLD"
text.lower() # 全部小寫: "hello world"
text.capitalize() # 首字母大寫: "Hello world"
text.title() # 每個單字首字母大寫: "Hello World"
text.swapcase() # 大小寫互換: "hELLO wORLD"
```

```
# String 可用方法 - 判斷
text = "Hello123"
text.isalpha() # 是否全為字母: False
text.isdigit() # 是否全為數字: False
text.isalnum() # 是否全為字母或數字: True
text.isspace() # 是否全為空白字元: False
text.isupper() # 是否全為大寫: False
text.islower() # 是否全為小寫: False
text.islower() # 是否為標題格式 (每個單字首字母大寫):
False
text.startswith("Hello") # 是否以指定字串開頭: True
text.endswith("123") # 是否以指定字串結尾: True
print("123".isdecimal()) # True
print("% ".isdecimal()) # False
print("123.45".isdecimal()) # False
print("123".isnumeric()) # True
print("% ".isnumeric()) # True - Unicode fraction
Unicode
print("Ⅰ".isnumeric()) # True - Roman numeral
```

```
# String 可用方法 - 搜尋
text = "Hello World Hello"
text.find("World") # 找到回傳索引位置, 找不到回傳 -1: 6
text.rfind("Hello") # 從右邊開始找: 12
text.index("World") # 找到回傳索引位置, 找不到引發
ValueError:6
```

```
text.rindex("Hello") # 從右邊開始找: 12
text.count("Hello") # 計算出現次數: 2
```

```
# String 可用方法 - 修改 (回傳新字串, 原字串不變)
text = " Hello World "
text.strip() # 移除前後空白: "Hello World"
text.lstrip() # 移除左邊空白: "Hello World "
text.rstrip() # 移除右邊空白: " Hello World"
text.strip("Hd ") # 移除指定字元: "ello Worl"
```

```
text = "Hello World"
text.replace("World", "Python") # 取代字串: "Hello
Python"
text.replace("l", "L", 2) # 取代前兩個: "HeLLO World"
```

```
# String 可用方法 - 分割與合併
text = "apple,banana,orange"
fruits = text.split(",") # 分割字串: ['apple', 'banana',
'orange']
text = "Hello World Python"
words = text.split() # 以空白分割: ['Hello', 'World', '
Python']
text = "apple,banana,orange,grape"
parts = text.split(",", 2) # 限制分割次數: ['apple', '
banana', 'orange,grape']
```

```
lines = "Line1\nLine2\nLine3"
line_list = lines.splitlines() # 以換行符號分割: ['
Line1', 'Line2', 'Line3']
```

```
## 合併字串
fruits = ["apple", "banana", "orange"]
text = ",".join(fruits) # 用逗號合併: "apple,banana,
orange"
text = " ".join(fruits) # 用空格合併: "apple banana
orange"
text = "".join(fruits) # 直接合併: "applebananaorange"
```

```
# String 可用方法 - 對齊與填充
text = "Hello"
text.center(10) # 置中, 寬度10: " Hello "
text.center(10, "*") # 用 * 填充: "***Hello***"
text.ljust(10) # 左對齊: "Hello "
text.ljust(10, "-") # 用 - 填充: "Hello-----"
text.rjust(10) # 右對齊: " Hello"
text.rjust(10, "*") # 用 * 填充: "*****Hello"
text.zfill(10) # 用 0 填充 (數字專用): "00000Hello"
```

```
# 字串索引與切片
text = "Hello World"
text[0] # 第一個字元: 'H'
text[-1] # 最後一個字元: 'd'
text[0:5] # 切片 (索引 0 到 4): "Hello"
text[6:] # 從索引 6 到結尾: "World"
text[:5] # 從開始到索引 4: "Hello"
text[:2] # 每隔一個字元: "HloWrld"
text[::-1] # 反轉字串: "dlroW olleH"
text[6:11] # "World"
# text[0] = 'h' # 錯誤! 字串不可變
```

```
# 跳脫字元
text = "Hello\nWorld" # 換行
text = "Hello\tWorld" # Tab
text = "He said \"Hello\"" # 雙引號
text = 'It\'s a book' # 單引號
text = "Path: C:\\Users\\name" # 反斜線
text = "Line1\rLine2" # 回車 (Carriage Return)
text = "Unicode: \u0041" # Unicode 字元: "Unicode: A"
```

```
# 字串運算
text1 = "Hello"
text2 = "World"
combined = text1 + " " + text2 # 串接: "Hello World"
repeated = text1 * 3 # 重複: "HelloHelloHello"
is_in = "ell" in text1 # 檢查子字串: True
not_in = "xyz" not in text1 # True
```

```
# 字串比較
"apple" == "apple" # 相等: True
"apple" != "banana" # 不相等: True
"apple" < "banana" # 字典序比較: True (依 ASCII/Unicode
)
```

```

"Apple" < "apple" # True (大寫字母 < 小寫字母)

# 字串迭代
text = "Hello"
for char in text:
    print(char) # 逐字元輸出

## 使用 enumerate 取得索引和字元
for i, char in enumerate(text):
    print(f"{i}: {char}")

# String to List
text = "Hello World"
char_list = list(text) # 轉換為字元列表: ['H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd']
words = text.split() # 分割為單字列表: ['Hello', 'World']

# List to String
chars = ['H', 'e', 'l', 'l', 'o']
text = "".join(chars) # "Hello"
words = ['Hello', 'World']
text = " ".join(words) # "Hello World"

# 常見應用範例
## 移除特定字元
text = "Hello, World!"
cleaned = text.replace(",", "").replace("!", "") # "Hello World"

## 統計字元出現次數
text = "Hello World"
count = text.count("l") # 3

## 檢查是否包含特定字串
if "World" in text:
    print("Found!")

## 分割 Email
email = "user@example.com"
username, domain = email.split("@") # username="user", domain="example.com"

## 處理路徑
path = "C:/Users/Alice/Documents/file.txt"
parts = path.split("/")
filename = parts[-1] # "file.txt"

## 反轉字串
text = "Hello"
reversed_text = text[::-1] # "olleH"

## 移除標點符號
import string
text = "Hello, World! How are you?"
cleaned = text.translate(str.maketrans("", "", string.punctuation)) # "Hello World How are you"

## 計算每個字元出現次數
text = "hello"
char_count = {}
for char in text:
    char_count[char] = char_count.get(char, 0) + 1
# {'h': 1, 'e': 1, 'l': 2, 'o': 1}

## 使用 Counter (更簡潔)
from collections import Counter
char_count = Counter(text) # Counter({'l': 2, 'h': 1, 'e': 1, 'o': 1})

## 檢查回文 (Palindrome)
def is_palindrome(text):
    text = text.lower().replace(" ", "")
    return text == text[::-1]

print(is_palindrome("A man a plan a canal Panama")) # True

## 移除多餘空白
text = "Hello World Python"
cleaned = " ".join(text.split()) # "Hello World Python"

```

```

## 首字母縮寫
phrase = "Portable Network Graphics"
acronym = "".join(word[0] for word in phrase.split()) # "PNG"

## 隱藏敏感資訊
credit_card = "1234-5678-9012-3456"
masked = "*" * 12 + credit_card[-4:] # "*****3456"

## 字串對齊輸出
items = ["Apple", "Banana", "Orange"]
for item in items:
    print(f"{item:<10} - {len(item):>2} chars")
# Apple      - 5 chars
# Banana     - 6 chars
# Orange     - 6 chars

## 字串正規化 (Normalization)
import unicodedata
text = "Café"
normalized = unicodedata.normalize('NFD', text) # 分解重音符號
ascii_text = normalized.encode('ascii', 'ignore').decode() # "Cafe"

## 產生隨機字串
import random
import string
length = 8
random_string = ''.join(random.choices(string.ascii_letters + string.digits, k=length))
# 例如: "aB3xY9kL"

## 多行字串處理
lines = """Line 1
Line 2
Line 3"""
line_list = [line.strip() for line in lines.split('\n') if line.strip()]
# ['Line 1', 'Line 2', 'Line 3']

```

2.6 堆疊 Stack

```

class Stack:
    def __init__(self):
        self.items = []
    def is_empty(self):
        return len(self.items) == 0
    def push(self, item):
        self.items.append(item)
    def pop(self):
        if self.is_empty():
            raise IndexError("Pop from empty stack")
        return self.items.pop()
    def peek(self):
        if self.is_empty():
            raise IndexError("Peek from empty stack")
        return self.items[-1]
    def size(self):
        return len(self.items)

```

2.7 佇列 Queue

```

from collections import deque
class Queue:
    def __init__(self):
        self.items = deque()
    def is_empty(self):
        return len(self.items) == 0
    def enqueue(self, item):
        self.items.append(item)
    def dequeue(self):
        if self.is_empty():
            raise IndexError("Dequeue from empty queue")
        return self.items.popleft()
    def front(self):
        if self.is_empty():
            raise IndexError("Front from empty queue")
        return self.items[0]

```

```
def size(self):
    return len(self.items)
```

2.8 鏈結串列 Link List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None
    def is_empty(self):
        return self.head is None
    def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
    def insert_at_end(self, data):
        new_node = Node(data)
        if self.is_empty():
            self.head = new_node
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node
    def delete(self, data):
        if self.is_empty():
            return
        if self.head.data == data:
            self.head = self.head.next
            return
        current = self.head
        while current.next and current.next.data != data:
            current = current.next
        if current.next:
            current.next = current.next.next
    def display(self):
        if self.is_empty():
            print("Empty list")
            return
        elements = []
        current = self.head
        while current:
            elements.append(str(current.data))
            current = current.next
        print("->".join(elements))
```

2.9 二元樹 Binary Tree

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self, root_data=None):
        self.root = TreeNode(root_data) if root_data else None
    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)
    def preorder(self, node):
        if node:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)
    def postorder(self, node):
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")
```

3 函式 Function

3.1 你該知道的事 Things You Should Know

預設參數 (default arguments)，當呼叫函式時沒有提供對應的參數值，則會使用預設參數值。但預設參數只會被評估一次，所以千萬不要放可變物件 (mutable objects) 當作預設參數值，否則會發生鬼故事。預設參數必須放在非預設參數的後面，否則會發生語法錯誤。

```
# 將可變物件作為預設參數
# 該物件會在函式定義時被建立一次，之後每次呼叫函式時都會使用同一個物件。
def add_to_list(item, target_list=[]):
    target_list.append(item)
    return target_list
print(add_to_list(1)) # [1]
print(add_to_list(2)) # [1, 2]
print(add_to_list(3)) # [1, 2, 3]

# 預期結果應該這麼寫
def add_to_list2(item, target_list=None):
    if target_list is None:
        target_list = []
    target_list.append(item)
    return target_list

print(add_to_list2(1)) # [1]
print(add_to_list2(2)) # [2]
print(add_to_list2(3)) # [3]
```

Python 的變數優先級：Local (區域) -> Enclosing (封閉) -> Global (全域) -> Built-in (內建)。- global：在函式內修改全域變數。- nonlocal：在巢狀函式中修改外層 (非全域) 變數。

用來判斷目前的模組是否是被直接執行，而不是被其他模組匯入 (import)：

```
if __name__ == "__main__":
```

3.2 位置參數與關鍵字參數 Positional and Keyword Arguments

```
# 位置參數 (Positional Arguments)
# 必須按照正確的順序提供參數
def introduce(name, age, city):
    print(f"Hi, I'm {name}, {age} years old, from {city}")
    introduce("Alice", 25, "Taipei")

# 關鍵字參數 (Keyword Arguments)
# 使用 name=value 的方式傳入，順序不拘。
def create_profile(name, age, city, occupation):
    """Create user profile using keyword arguments"""
    print(f"Profile: {name}, {age}, {city}, {occupation}")

create_profile(name="Bob", city="Kaohsiung", age=30,
               occupation="Engineer")
create_profile(age=28, name="Carol", occupation="Designer", city="Taichung")
# 混合使用位置參數與關鍵字參數，這時關鍵字參數必須放在位置參數之後
create_profile("David", 35, city="Tainan", occupation="Teacher")
```

3.3 可變長度參數 Variable-length Arguments

*args: 將傳入的多個「位置參數」打包成一個 tuple 傳給 Function。*kwargs: 將傳入的多個「關鍵字參數」打包成一個 dict 傳給 Function。

```
# *args 範例：允許傳入任意數量的數字來加總
# *args 會將傳入的參數打包成一個 tuple (永遠都是 tuple)
def calculate_sum(*numbers):
    """Calculate sum of any number of arguments"""
    print(f"Received arguments: {numbers}") # numbers 是一個 tuple
    total = sum(numbers)
    return total

print(calculate_sum(1, 2, 3)) # Received arguments: (1, 2, 3) -> 6
print(calculate_sum(10, 20, 30, 40)) # Received arguments: (10, 20, 30, 40) -> 100

# **kwargs 範例：允許傳入任意數量的關鍵字屬性
```

```
# **kwargs 會將傳入的參數打包成一個 dict (永遠都是 dict)
def create_user_profile(name, **details):
    """Create user profile with flexible details"""
    print(f"User: {name}")
    print("Details:")
    # details 是一個 dict
    for key, value in details.items():
        print(f" {key}: {value}")

create_user_profile("Alice", age=25, city="Taipei", job="Engineer")
create_user_profile("Bob", age=30, country="Taiwan")
```

3.4 高階函式 Higher-Order Functions

```
# 先定義幾個簡單的操作函式
def square(x):
    return x**2
def double(x):
    return x * 2

# Higher Order Function
# 接受一個列表和一個「操作函式」
def apply_operation(numbers, operation):
    """Apply an operation to all numbers in a list"""
    result = []
    for num in numbers:
        result.append(operation(num)) # 呼叫傳入的
        operation 函式
    return result

numbers = [1, 2, 3, 4, 5]

# 傳入 square 函式
print(apply_operation(numbers, square)) # 輸出: [1, 4, 9, 16, 25]
# 傳入 double 函式
print(apply_operation(numbers, double)) # 輸出: [2, 4, 6, 8, 10]
```

3.5 閉包 Closure

```
# 這是一個高階函式，它回傳一個新的函式
def create_multiplier(factor):
    """Create a function that multiplies by a specific factor"""

    # create_multiplier 要回傳的內部函式
    def multiplier(number):
        return number * factor

    return multiplier # 回傳內部函式

# 建立一個「乘以 2」的函式
double = create_multiplier(2)
# 建立一個「乘以 3」的函式
triple = create_multiplier(3)

# 使用新建立的函式
print(double(5)) # 輸出: 10
print(triple(4)) # 輸出: 12
```

3.6 Lambda 函式 Lambda Functions

```
# 一般函式
def add(x, y):
    return x + y

# Lambda 函式
add_lambda = lambda x, y: x + y

print(add(5, 3)) # 輸出: 8
print(add_lambda(5, 3)) # 輸出: 8

# 單一參數的範例
square_lambda = lambda x: x**2
print(square_lambda(4)) # 輸出: 16

# 將 lambda 作為參數傳遞 - 用作排序時
students = [
    {'name': 'Alice', 'age': 23, 'grade': 88},
```

```
{'name': 'Bob', 'age': 21, 'grade': 92},
{'name': 'Carol', 'age': 22, 'grade': 85}]
# 使用 lambda 指定以 'age' 排序
students_by_age = sorted(students, key=lambda student:
    student['age'])
```

3.7 內建高階函式 Built-in Higher-Order Functions

- map(function, iterable): 將一個函式應用到一個 (或多個) 可迭代物件 (iterable) 的每一個元素上，並回傳一個迭代器。
- filter(function, iterable): 根據一個函式 (該函式回傳 True 或 False) 來過濾一個可迭代物件，只保留使 function 回傳 True 的元素，並回傳一個 filter 物件 (可轉換成 list)。
- reduce(function, iterable): 將一個可迭代物件中的元素，使用一個函式 (該函式接受兩個參數) 進行「累加」計算，最終回傳一個單一的值。

```
# map usage
## 將數字平方
numbers = [1, 2, 3, 4, 5]
### 使用 map 和 lambda 將 numbers 中的每個元素都平方
squared = list(map(lambda x: x**2, numbers))
print(squared)
### 輸出: [1, 4, 9, 16, 25]

## 攝氏溫度轉華氏
celsius_temps = [0, 20, 30, 40, 100]
### 套用溫度轉換公式
fahrenheit_temps = list(map(lambda c: c * 9/5 + 32,
    celsius_temps))
print(fahrenheit_temps)
### 輸出: [32.0, 68.0, 86.0, 104.0, 212.0]

# filter usage
## 篩選出偶數
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
### 使用 filter 和 lambda 保留 x % 2 == 0 (偶數) 的元素
even_numbers = list(filter(lambda x: x % 2 == 0,
    numbers))
print(even_numbers)
#### 輸出: [2, 4, 6, 8, 10]

## 篩選出長度大於 5 的字串
strings = ["apple", "banana", "cherry", "date"]
long_strings = list(filter(lambda s: len(s) > 5,
    strings))
print(long_strings)
### 輸出: ['banana', 'cherry']

# reduce usage
from functools import reduce
## 將列表中的所有數字加總
numbers = [1, 2, 3, 4, 5]
'''
reduce 會這樣運作:
1 + 2 = 3
3 + 3 = 6
6 + 4 = 10
10 + 5 = 15
'''
total = reduce(lambda x, y: x + y, numbers)
print(total)
### 輸出: 15

## 找出列表中的最大值
numbers = [1, 2, 3, 4, 5]
### 比較 x 和 y，回傳較大者
maximum = reduce(lambda x, y: x if x > y else y,
    numbers)
print(maximum)
```

3.8 裝飾器 Decorators

裝飾器 (Decorator) 是一種設計模式，允許在不改變函式原始碼的情況下，為函式添加額外的功能。裝飾器本質上是高階函式 (Higher-Order Function)，接受一個函式作為參數，並回傳一個新的函式。可以想像成.....

- 你的函式：你寫好了一個函式，這是「禮物」。例如，一個計算階乘的函式。
- 裝飾器：這是一張「包裝紙」。這張包裝紙本身也是一個函式，它定義了「在打開禮物前要做的事」和「在打開禮物後要做的事」。

- @ 符號：這個符號就是「包裝動作」。當你把 @xxxxxx 放在你的函式上面時，你就等於是說：「嘿，Python，請用這張『xxxxxx 包裝紙』把我的『禮物函式』包起來。」
- 最終，當我們呼叫被裝飾過的函式時，實際上執行的是 wrapper 函式，而不是原始的 function (my_decorator 或 timing_decorator)。

```
# 步驟 1: 定義裝飾器 (包裝紙)
def my_decorator(func): # func 是被包裝的原始函式 (禮物)
    def wrapper(*args, **kwargs): # wrapper 是包裝後的新函式
        print(f"Before calling {func.__name__}") # 打開禮物前
        result = func(*args, **kwargs) # 打開禮物 (執行原始函式)
        print(f"After calling {func.__name__}") # 打開禮物後
        return result
    return wrapper # 回傳包裝好的新函式
```

```
# 步驟 2: 套用裝飾器 (包裝動作)
@my_decorator
def greet(name):
    print(f"Hello, {name}!")
    return f"Greeting for {name}"
```

```
# 步驟 3: 呼叫 (你呼叫的是「包裝後」的函式)
result = greet("Alice")
print(f"Result: {result}")
```

```
# 輸出:
# Before calling greet
# Hello, Alice!
# After calling greet
# Result: Greeting for Alice
```

```
import time
import functools # 用於 @functools.wraps

def timing_decorator(func):
    """Decorator to measure function execution time"""

    # @functools.wraps 會保留原始函式的名稱等資訊，是個好習慣
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start_time = time.time() # 執行前：記錄開始時間
        result = func(*args, **kwargs) # 執行原始函式
        end_time = time.time() # 執行後：記錄結束時間

        execution_time = end_time - start_time # 計算耗時
        print(f"{func.__name__} executed in {execution_time:.4f} seconds")
        return result
    return wrapper

# 套用
@timing_decorator
def slow_function():
    """Simulate a slow function"""
    time.sleep(1) # 模擬耗時 1 秒 [cite: 1779]
    return "Done!"

print(slow_function())

# 輸出 (時間會接近 1.0000):
# slow_function executed in 1.0002 seconds
# Done!
```

3.9 迭代器 Iterators

迭代器是一種物件，它實現了迭代協定 (即 `__iter__()` 和 `__next__()` 方法)，可以用來遍歷容器 (如 `list`、`tuple`、`dict` 等) 的元素。Iterable (可迭代物件)：像是 `list`，你可以從它身上取得迭代器。Iterator (迭代器)：是 `iter()` 函式回傳的物件，它會記住當前位置，並可用 `next()` 取出下一個值。

```
# 1. my_list 是一個「可迭代物件」(Iterable)
my_list = [1, 2, 3]
# 2. 使用 iter() 從可迭代物件中取得「迭代器」(Iterator)
my_iterator = iter(my_list)
```

```
print(f"這是一個迭代器物件: {my_iterator}")
```

```
# 3. 使用 next() 從迭代器中取出下一個值
print(next(my_iterator)) # 輸出: 1
print(next(my_iterator)) # 輸出: 2
print(next(my_iterator)) # 輸出: 3
```

```
# 4. 當所有值都取完後，再次呼叫 next() 會引發 StopIteration 錯誤
# print(next(my_iterator)) # 這行會引發 StopIteration
```

Python 中有很多內建的物件都是可迭代的，甚至本身就是迭代器。

```
# 1. Range 物件
print("\n--- Range ---")
for i in range(3): # range(3) 是一個可迭代物件
    print(i)
# 輸出: 0, 1, 2
```

```
# 2. 字典 (Dictionary) 的 views
print("\n--- 字典 ---")
my_dict = {'a': 1, 'b': 2, 'c': 3}
```

```
# .keys() .values() .items() 都是可迭代的
print("Keys:")
for key in my_dict.keys(): # 迭代所有鍵 (key)
    print(key)
```

```
print("Values:")
for value in my_dict.values(): # 迭代所有值 (value)
    print(value)
```

```
print("Items:")
for item in my_dict.items(): # 迭代所有 (鍵, 值) 對
    print(item)
```

3.10 產生器 Generators

產生器 (Generator) 是一種特殊類型的迭代器，用於在需要時動態生成值，而不是一次性將所有值存儲在記憶體中。產生器與 `return` 最本質的差別如下：

- `return`：像一張「單程車票」。當函式執行到 `return` 時，函式會結束並回傳一個值，之後無法再從該函式繼續執行。
- `yield`：像一個「遊戲存檔點」。當函式執行到 `yield` 時，函式會暫停並回傳一個值，但保留其狀態，儲存它所有的狀態 (記住 `a`, `b`, `count` 這些變數現在的值)，以便下次從暫停的地方繼續執行。當你要求下一個值時 (呼叫 `next()`)，函式會從上次凍結的地方「解除暫停」，繼續執行，直到遇到下一個 `yield`。

```
def fibonacci_generator(max_count=None):
    a, b = 0, 1
    # 初始化前 fib 的兩個數字
    count = 0

    # 迴圈會一直執行，直到達到 max_count
    while max_count is None or count < max_count:
        yield a # 回傳目前的 Fibonacci 數字，然後暫停函式執行
        a, b = b, a + b # 計算下一個數字
        count += 1
```

```
print("--- First 10 Fibonacci numbers: ---")
# for 迴圈會自動處理 next() 呼叫和 StopIteration
for i, fib in enumerate(fibonacci_generator(10)):
    print(f"F({i}) = {fib}")
```

```
# 輸出:
# F(0) = 0
# F(1) = 1
# F(2) = 1
# F(3) = 2
# F(4) = 3
# ... (依此類推)
```

4 File Handling

4.1 基本檔案操作 Basic File Operations

檔案開啟的模式可以分成：

- `r`：唯讀 (預設)
- `w`：寫 (覆蓋原有檔案)
- `a`：附加 (在檔案末尾新增內容)

- x: 建立新檔案 (檔案已存在會失敗)
- b: 二進位 (與其他模式結合使用, 如 rb, wb)
- t: 文字模式 (預設, 與其他模式結合使用, 如 rt, wt)
- +: 讀寫模式 (與其他模式結合使用, 如 r+, w+)

```
# 寫檔
with open('out.txt', 'w') as f:
    f.write("Hello\n")
```

```
# 讀檔
with open('in.txt', 'r') as f:
    content = f.read()
    print(content)
```

5 Exception Handling

5.1 基本例外處理 Basic Exception Handling

常見的內建例外類型：

- ValueError: 傳入無效的值 (例如 `int("abc")`)。
- TypeError: 對不相容的型別進行操作 (例如 `"5" + 5`)。
- IndexError: 使用了無效的索引值。
- KeyError: 使用了字典中不存在的鍵。
- ZeroDivisionError: 除以零。
- FileNotFoundError: 試圖開啟不存在的檔案。
- AttributeError: 試圖存取物件沒有的屬性或方法。
- EOFError: 檔案讀取到結尾。

```
try:
    x = int(input("Enter a number: "))
    print("You entered:", x)
except Exception as e:
    print("An error occurred:", e)
```

5.2 else 和 finally 子句 else and finally Clauses

- 'else' 子句: 只有在 try 區塊中「沒有」發生任何例外時, else 區塊的程式碼才會被執行。
- 'finally' 子句: 無論是否發生例外 (無論是成功、發生錯誤被捕捉、或發生錯誤未被捕捉), finally 區塊的程式碼「永遠」會被執行。

```
try:
    # 1. 嘗試執行
    result = 10 / 2
except ZeroDivisionError:
    # 2. 如果發生 ZeroDivisionError, 執行這裡
    print("Error: Division by zero!")
else:
    # 3. 如果「沒有」發生例外, 執行這裡
    print(f"Division successful! Result: {result}")
finally:
    # 4. 「無論如何」都會執行這裡
    print("Division operation completed.")
```

5.3 自訂例外 Custom Exceptions

可以透過繼承內建的 Exception 類別來建立自訂的例外類型。

```
class AgeNotLegalError(Exception):
    pass

age = -5
if age < 0:
    raise AgeNotLegalError("Age cannot be negative.")
'''
Traceback (most recent call last):
File "/home/main.py", line 6, in <module>
    raise AgeNotLegalError("Age cannot be negative.")
AgeNotLegalError: Age cannot be negative.
'''
```

5.4 斷言 Assertions

assert 用來檢查一個條件是否為 True。如果條件為 False, assert 會引發 AssertionError 例外。assert 通常用被拿來當作程式碼中的內部檢查點 (internal checkpoints), 以確保程式在執行過程中符合預期的狀態。assert 的錯誤用法：

- 驗證使用者輸入
- 驗證外部資料 (如檢查檔案是否存在)
- 取代正常的錯誤處理邏輯
- 在生產環境中依賴 assert 來維護程式邏輯

```
# 帶有錯誤訊息的 assert
age = 17
assert age >= 18, "Age must be 18 or older"
print("Age check passed!")

'''
Traceback (most recent call last):
  File "/home/main.py", line 2, in <module>
    assert age >= 18, "Age must be 18 or older"
    ^^^^^^^^^^^^^
AssertionError: Age must be 18 or older
'''

# 沒有錯誤訊息的 assert
x = 10
assert x > 0 # 條件為 True, 靜默通過
assert x < 5 # 條件為 False, 引發 AssertionError

'''
Traceback (most recent call last):
  File "/home/main.py", line 3, in <module>
    assert x < 5 # 條件為 False, 引發 AssertionError
    ^^^^^
AssertionError
'''
```

6 Object-Oriented

6.1 建立類別 Class Declaration

```
class Dog:
    # 類別屬性 (所有實例共享)
    species = "Canis familiaris"

    # 建構子: 建立實例屬性 (各自獨立)
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # 實例方法
    def bark(self):
        return f"{self.name} says Woof!"
```

6.2 類別的方法 Class Methods

```
class Demo:
    class_var = "Class Var"

    def __init__(self, v):
        self.instance_var = v

    # 實例方法要 self
    def instance_method(self):
        return f"instance -> {self.instance_var}, class -> {self.class_var}"

    # 類別方法用 cls 操作類別層資料
    @classmethod
    def class_method(cls):
        return f"class -> {cls.class_var}"

    # 靜態方法不能碰 self/cls
    @staticmethod
    def static_method(x):
        return f"static -> {x}"
```

6.3 建構子跟解構子 Constructor and Destructor

```
class sampleClass:
    def __init__(self):
        # init 在物件創建時就會執行
        # 除了 self 之外 init 要接多少參數就要傳多少過去
        print("Resource allocated")
    def __del__(self):
        # del 在物件被銷毀前執行
        print("Resource released")

obj = sampleClass() # 建立物件時會呼叫 __init__
del obj             # 刪除物件時會呼叫 __del__
```

6.4 封裝 Encapsulation

```
class BankAccount:
    def __init__(self, account_number, balance):
        self.account_number = account_number # public
        self._balance = balance              # protected
        self.__pin = "1234"                  # private

    def get_balance(self):                    # public
        return self._balance

    def __verify_pin(self, pin):              # private
        return pin == self.__pin
```

6.5 封裝屬性 Encapsulation with Property

@property 裝飾器 (Decorator) 可以把方法轉換成屬性。比如說, 'obj.x' 像是在讀某個值, 但實際上是呼叫 'obj.x()' (getter) 方法來取得值。在比如說, 'obj.x = value' 像是在設定某個值, 但實際上是呼叫 'obj.x(value)' (setter) 方法來設定值。

```
class Person:
    def __init__(self, age: int):
        self._age = None # 真正儲存在「私有欄位」
        self.age = age   # 走 setter 做一次驗證

    @property
    def age(self) -> int: # getter: 讀取時會被呼叫
        return self._age

    @age.setter
    def age(self, v: int): # setter: 賦值時會被呼叫
        if not isinstance(v, int):
            raise TypeError("age 必須是 int")
        if v < 0 or v > 150:
            raise ValueError("age 超出合理範圍")
        self._age = v

    @property
    def birth_year(self): # 計算屬性
        from datetime import datetime
        return datetime.now().year - self._age

    @property
    def is_adult(self): # 唯讀
        return self._age >= 18
```

```
p = Person(20)
p.age # -> 20 (call getter)
p.age = 30 # -> 30 (call setter)
p.age = -5 # ValueError: age 超出合理範圍
```

6.6 繼承 Inheritance

基本繼承:

```
class Animal:
    def __init__(self, name, age):
        self.name, self.age = name, age
    def make_sound(self): return "Some generic sound"
    def info(self): return f"{self.name} is {self.age} years old"
```

```
class Dog(Animal):
    def __init__(self, name, age, breed):
        super().__init__(name, age) # 呼叫父類建構子
        self.breed = breed
    def make_sound(self): return "Woof! Woof!" # 覆寫
    def fetch(self): return f"{self.name} fetching!"
```

```
class Cat(Animal):
    def __init__(self, name, age, color):
        super().__init__(name, age) # 呼叫父類建構子
        self.color = color

    def make_sound(self): return "Meow! Meow!" # 覆寫
    def scratch(self): return f"{self.name} is scratching!"
```

```
dog = Dog("Buddy", 3, "Golden Retriever")
cat = Cat("Whiskers", 2, "Gray")
```

多重繼承:

```
class Flyable: # mixin
    def fly(self): return f"{self.name} is flying!"

class Swimmable: # mixin
    def swim(self): return f"{self.name} is swimming!"

class Animal:
    def __init__(self, name): self.name = name
    def eat(self): return f"{self.name} is eating"
```

```
class Duck(Animal, Flyable, Swimmable):
    def __init__(self, name): super().__init__(name)
    def quack(self): return f"{self.name} says Quack!"
```

```
duck = Duck("Donald")
print(duck.eat()) # From Animal
print(duck.fly()) # From Flyable
print(duck.swim()) # From Swimmable
print(duck.quack()) # From Duck
```

6.7 多型 Polymorphism

```
class Shape:
    def area(self): return 0
    def perimeter(self): return 0

class Rectangle(Shape):
    def __init__(self, w, h): self.w, self.h = w, h
    def area(self): return self.w * self.h
    def perimeter(self): return 2 * (self.w + self.h)

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self):
        import math
        return math.pi * self.r ** 2
    def perimeter(self):
        import math
        return 2 * math.pi * self.r
```

7 Algorithm

7.1 矩陣操作 Matrix Operations

```
# 矩陣轉置
matrix_transpose = [[matrix[j][i] for j in range(len(matrix)) for i in range(len(matrix[0]))]]

# 水平翻轉 (上下顛倒)
j = len(matrix) - 1
for i in range(len(matrix)//2):
    matrix[i], matrix[j] = matrix[j], matrix[i]
    j -= 1

# 垂直翻轉 (左右顛倒)
j = len(matrix) - 1
for i in range(len(matrix)):
    matrix[i][0], matrix[i][j] = matrix[i][j], matrix[i][0]

# 逆時鐘轉 90 度
```

```
matrix_reverse = [ row[::-1] for row in matrix ]
matrix = [[matrix_reverse[j][i] for j in range(len(
    matrix_reverse))] for i in range(len(matrix_reverse
    )) ]
```

順時鐘轉 90 度

```
matrix_transpose = [[ matrix[j][i] for j in range(len(
    matrix))] for i in range(len(matrix[0]))]
matrix = [ row[::-1] for row in matrix_transpose ]
```

7.2 Sort: Bubble Sort

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
```

7.3 Sort: Selection Sort

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

7.4 Sort: Insertion Sort

```
def insertion_sort(arr):
    n = len(arr)

    for i in range(1, n):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
```

7.5 Sort: Quick Sort

```
def qsort( arr , l , r ):
    if l >= r:
        return
    i, j = l, r
    pivot = arr[(l+r)//2]
    while i <= j:
        while arr[i] < pivot:
            i += 1
        while arr[j] > pivot:
            j -= 1
        if i <= j:
            tp = arr[i]
            arr[i] = arr[j]
            arr[j] = tp
            i += 1
            j -= 1

    qsort( arr, l, j )
    qsort( arr, i, r )
```

```
qsort(num, 0, len(num) - 1 )
#num is a list
```

7.6 Sort: Quick Sort(In Place)

```
def partition(arr, low, high):
    pivot = arr[high]
    idx = low
    for i in range(low, high):
        if arr[i] < pivot:
            if i != idx:
                tp = arr[i]
                arr[i] = arr[idx]
                arr[idx] = tp
            idx += 1
```

```
if idx != high:
    tp = arr[idx]
    arr[idx] = arr[high]
    arr[high] = tp
return idx
```

```
def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)
```

```
quick_sort(num, 0, len(num)-1)
#num is a list
```

7.7 Sort: Merge Sort

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = arr[:mid]
    right_half = arr[mid:]

    sorted_left = merge_sort(left_half)
    sorted_right = merge_sort(right_half)

    merged_list = []
    i = 0
    j = 0

    while i < len(sorted_left) and j < len(sorted_right):
        if sorted_left[i] < sorted_right[j]:
            merged_list.append(sorted_left[i])
            i += 1
        else:
            merged_list.append(sorted_right[j])
            j += 1

    while i < len(sorted_left):
        merged_list.append(sorted_left[i])
        i += 1

    while j < len(sorted_right):
        merged_list.append(sorted_right[j])
        j += 1
```

7.8 Search: Binary Search

```
def binary_search(arr, tar) -> int:
    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == tar:
            return mid
        elif arr[mid] < tar:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

7.9 String: KMP

```
def getLps( tar ):
    lps = [0] * len(tar)
    prefix = 0
    i = 1
    while i < len(tar):
        if tar[prefix] == tar[i]:
            prefix += 1
            lps[i] = prefix
            i += 1
        else:
            if prefix != 0:
                prefix = lps[prefix - 1]
```

```
        else:
            lps[i] = 0
            i += 1
    return lps

def kmp( orig_str, tar ):
    if not tar:
        return []
    lps = getLps(tar)
    ans = list()
    origPtr = 0
    targPtr = 0
    while origPtr < len(orig_str):
        if orig_str[origPtr] == tar[targPtr]:
            origPtr += 1
            targPtr += 1

        if targPtr == len(tar):
            ans.append(origPtr - targPtr)
            targPtr = lps[targPtr-1]
        elif origPtr < len(orig_str) and orig_str[
            origPtr] != tar[targPtr]:
            if targPtr != 0:
                targPtr = lps[targPtr-1]
            else:
                origPtr += 1
    # return all appear index(a list)
    return ans
```